

GraphReduce: Relational Signal Compression via Interoperable Graph-Based Feature Execution

Wes Madrigal
Kurve, Inc.
wes@kurve.ai

Abstract

Machine learning over enterprise data still depends on an expensive and failure-prone preprocessing step: transforming many heterogeneous relational tables into a single leakage-safe training frame at the correct parent grain. Existing automated feature engineering systems showed that some of this work can be abstracted, but they often remain constrained by a particular execution substrate, offer limited customization, or underemphasize temporal correctness and graph-shaped relational structure. This paper presents GraphReduce, a programming model and software abstraction for *relational signal compression*: the process of converting predictive signal distributed across many related tables into a compact, model-ready representation at a chosen parent grain. GraphReduce models datasets as nodes, relationships as edges, and execution as a deterministic sequence of abstract node- and graph-level operations. Developers may implement these operations directly or delegate parts of them to dynamic feature-generation behavior, allowing a continuum between fully custom feature logic and automated feature engineering. GraphReduce separates logical feature construction from physical execution, enabling the same computational graph to run across dataframe engines and SQL warehouses. The framework also treats point-in-time correctness as a first-class property by pushing shared temporal constraints through the graph, allowing the same graph to be regenerated across arbitrary historical windows while preserving leakage safety. Across a 21 task RelBench summary assembled from the KumoRFM-2 benchmark paper and the current GraphReduce plus CatBoost run suite, GraphReduce remains competitive with relational deep learning and relational foundation model baselines despite relying on a fundamentally different mechanism: deterministic relational signal compression into a leakage-safe parent-grain frame. On the 19 tasks for which GraphReduce results are available, GraphReduce plus CatBoost outperforms KumoRFM-1 on 12 tasks and KumoRFM-2 on 8 tasks, while remaining fully customizable, immediately usable in production, and directly compatible with existing MLOps infrastructure and arbitrary downstream model choices. Source code and project materials are available at <https://github.com/wesmadrigal/GraphReduce>.

1 Introduction

In enterprise settings, predictive signal is rarely contained in a single table. Instead, it is distributed across customers, orders, products, notifications, events, logs, interactions, and many other entities stored at different grains and connected through keys and time. Downstream models, however, usually require a single dataset at a single grain. Bridging that mismatch requires repeated implementation of joins, aggregations, filters, normalization, time-window logic, label construction, and leakage prevention.

This problem is structural rather than cosmetic. Relational datasets are multi-granular, temporally constrained, and often too complex to flatten manually without introducing duplication, information

loss, or future leakage. The resulting pipelines are frequently bespoke, hard to inspect, tightly coupled to one compute engine, and difficult to rerun consistently across historical windows.

GraphReduce is motivated by the observation that many of these pipelines share a stable high-level computational pattern. Tables can be represented as nodes, relationships as edges, and feature generation as a traversal over a graph in which child tables are reduced and joined back to a parent grain. GraphReduce provides an abstraction layer over that pattern.

The project is inspired, in part, by the simplifying spirit of MapReduce [1]. MapReduce exposed a compact programming model that separated logical intent from the complexity of distributed execution. GraphReduce pursues a similar separation, but for a different domain: relational feature generation, cardinality-aware rollups, and leakage-safe data preparation rather than generic distributed batch processing. The comparison is therefore conceptual rather than literal.

This paper argues that GraphReduce should be understood not only as an automated feature engineering library, but more broadly as a system for *relational signal compression*. That phrase captures the central practical task: compressing predictive signal distributed across a relational graph into a single representation appropriate for modeling, scoring, or downstream analytics.

2 Problem Setting

Consider an enterprise schema with entities such as customers, orders, order line items, notifications, and interaction events. Suppose the goal is to predict a customer-level outcome such as churn, next-period purchase count, or response propensity. The relevant predictive information is distributed across multiple tables, each with its own key structure, cardinality, update cadence, and time semantics.

The required output is a dataset with one row per customer and columns summarizing predictive signal from all relevant child entities under a leakage-safe temporal boundary. Producing that dataset typically involves at least the following recurring operations: filtering each source table, computing per-table annotations, normalizing or cleaning data, aggregating lower-grain entities to higher-grain parents, joining reduced child outputs into parents, computing post-join transformations requiring multi-table context, and generating labels using a future window that is disjoint from the feature window.

These operations recur across tasks and domains. What changes are the graph topology, grain, target definition, temporal windowing, compute backend, and the extent to which the logic is handwritten versus automated.

3 Relational Signal Compression

The literature commonly uses the term *automated feature engineering* to describe systems that automatically generate candidate features from structured data. GraphReduce clearly participates in that tradition [2, 3, 4].

We use the term *relational signal compression* to refer to the broader process of transforming predictive information spread across a set of related tables into a compact representation at the grain required by a downstream task. This framing emphasizes three things. First, the signal is distributed across multiple entities and relationship paths. Second, compression is grain-specific: the desired output is a single parent-grain dataset rather than a collection of isolated feature tables.

Third, the operation is constrained by both cardinality and time: a valid compression must preserve intended grain, avoid duplicate amplification, and respect point-in-time boundaries.

Under this framing, automated feature engineering becomes one mechanism by which relational signal compression can be achieved. GraphReduce supports both automated and custom pathways.

4 Design Goals

4.1 Stable abstraction over recurring operations

The system should expose a predictable sequence of operations that mirrors how multi-table feature engineering is already performed in practice.

4.2 Interoperability across execution substrates

The same logical graph should be able to run across dataframe systems and SQL warehouses without rewriting the entire feature pipeline from scratch.

4.3 Support for both automation and customization

Pure automation is attractive initially, but production systems frequently require custom filtering, entity-specific aggregation, deduplication, anomaly handling, or domain-specific enrichment. The abstraction should support both extremes and the space between them.

4.4 Point-in-time correctness as a first-class property

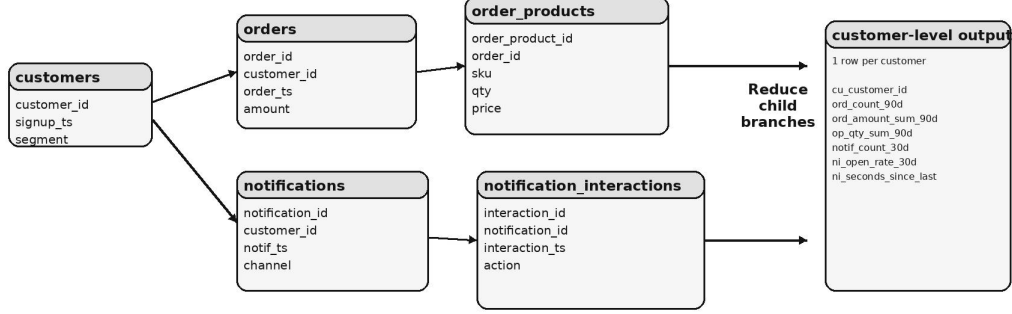
Temporal correctness must not be a bolt-on concern. It should be embedded into the graph-level execution semantics so that the same computational graph can be rerun at arbitrary windows without rewriting business logic.

5 System Overview

GraphReduce represents a relational dataset as a directed graph. Nodes correspond to entity tables or intermediate datasets. Edges correspond to join relationships between entities. A designated root or parent node defines the grain to which the graph is reduced. Graph-level parameters specify shared execution semantics such as compute backend, cut date, feature lookback window, label horizon, and optional target configuration.

A GraphReduce program therefore consists of three main ingredients: (1) a graph object that stores the relational topology and execution configuration, (2) a collection of node objects that define dataset-specific operations, and (3) a set of edges that specify how child entities relate to parents and whether a child must be reduced before joining. This representation turns relational structure into an executable plan. Instead of reasoning about one long procedural script, the developer reasons about entities, relationships, and a standard operation contract.

figure 1. Relational signal compression from graph to parent-grain frame



GraphReduce traverses the relational graph, reduces lower-grain entities along each branch, and joins the resulting summaries into a single parent-grain training frame. Prefixes preserve table lineage after merge and reduction.

Figure 1: GraphReduce performs relational signal compression by traversing a relational graph, reducing lower-grain child entities along each branch, and joining the resulting summaries into a single parent-grain training frame. The prefixed output columns preserve table lineage after merge and reduction.

6 Execution Model

The execution model is the core contribution of GraphReduce. The system is built around a predictable per-node operation sequence combined with a graph-level traversal strategy.

6.1 Node-level operation contract

Each node conforms to the same abstract interface. The principal methods include `do_filters`, `do_annotate`, `do_normalize`, and `do_reduce`, along with `do_labels`, `do_post_join_annotate`, and `do_post_join_filters`.

This contract is important for two reasons. First, it gives the developer a stable mental model for where different kinds of logic belong. Second, it creates a consistent abstraction boundary across custom nodes, dynamic nodes, dataframe execution, and SQL execution.

6.2 Graph-level flow

At runtime, GraphReduce traverses the relationship graph in a depth-first fashion and executes a deterministic sequence: child-table preprocessing through `do_annotate`, `do_filters`, and `do_normalize`; child reduction through `do_reduce` when the corresponding edge requires reduction; join of the reduced child output into the parent; execution of parent post-join methods such as

`do_post_join_annotate` and `do_post_join_filters`; and optional label computation through `do_labels` using the configured label horizon.

This execution model makes relational cardinality explicit. Instead of joining arbitrary tables and then trying to repair duplicate amplification later, GraphReduce treats reduction as an intentional step in the traversal process.

Figure 2. Stable abstract execution contract

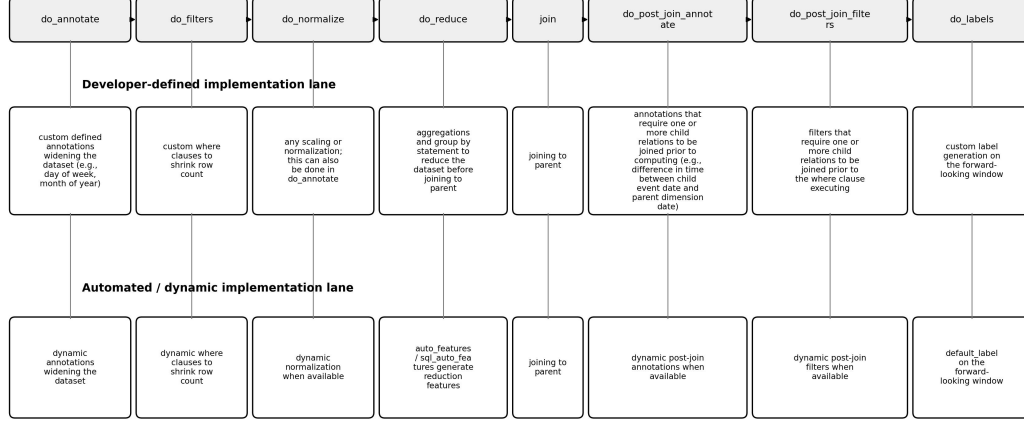


Figure 2: GraphReduce preserves a stable abstract execution contract independent of implementation strategy. Each node participates in the same logical sequence of operations, while the underlying method bodies may be implemented manually by the developer or synthesized dynamically for automated feature generation.

6.3 Execution of `do_transformations` and `do_transformations_sql`

The conceptual execution model in GraphReduce is reflected directly in the implementation of `do_transformations` and `do_transformations_sql`. Let the computational graph be a directed graph $G = (V, E)$ rooted at a parent node s , where each edge $e = (p, r)$ connects a parent node p to a relation node r . Each edge additionally carries a parent key, a relation key, a reduction flag, and optional post-join reduction metadata. At a high level, execution proceeds in three stages: (i) graph hydration and prefix validation, (ii) node-local preprocessing, and (iii) depth-first bottom-up traversal from the parent node [6].

Let D_v denote the relation stored at node $v \in V$. Let A_v , F_v , and N_v denote the node’s annotation, filtering, and normalization operators, respectively. Initialization first pushes graph-level attributes to nodes, hydrates graph data, verifies prefix uniqueness, and performs local preprocessing according to

$$D_v \leftarrow N_v(F_v(A_v(D_v))), \quad \forall v \in V. \quad (1)$$

If front traversal for automated features is enabled, GraphReduce may first traverse upward

from the parent node for a bounded number of hops and perform direct joins before the main reduction pass. After this optional stage, the principal execution order is obtained from a depth-first traversal rooted at s , with edges processed in reverse DFS order so that lower-grain descendants are materialized before their ancestors. This reverse traversal is important because it enforces a bottom-up reduction discipline: descendants are summarized before they are joined upward, which helps preserve cardinality in one-to-many relational paths [6].

Algorithm 1 summarizes the core execution pattern shared by `do_transformations` and `ndo_transformations_sql`. A key detail is that reduction and label generation each admit both custom and default execution paths. If a custom `do_reduce` implementation is present on a relation node and $\rho_e = 1$, GraphReduce executes that custom reduction on the point-in-time valid feature slice, i.e., after applying `prep_for_features`. Otherwise, when $\rho_e = 1$, it falls back to `auto_features` to synthesize the relation summary. The same pattern holds for labels: if a custom `do_labels` method is defined, that method is used; otherwise GraphReduce invokes `default_label` after applying `prep_for_labels` so that the label is computed only over the forward-looking label window.

Algorithm 1 Core data integration traversal for `do_transformations` and `do_transformations_sql`

Require: Directed graph $G = (V, E)$ rooted at parent node s

Require: For each edge $e = (p, r)$: parent key k_p , relation key k_r , reduction flag ρ_e , and post-join reduction flag α_e

Require: Optional automated feature settings and optional label node

```
1: Hydrate graph attributes and graph data
2: Check prefix uniqueness across nodes
3: for all  $v \in V$  do
4:    $D_v \leftarrow \text{do\_normalize}(\text{do\_filters}(\text{do\_annotate}(D_v)))$ 
5: end for
6: if front auto-feature traversal is enabled then
7:   for all upward edges within hop limit  $h_f$  do
8:     Join source node directly into target node
9:   end for
10: end if
11:  $E_{\text{dfs}} \leftarrow \text{reverse}(\text{dfs\_edges}(G, \text{source} = s, \text{depth\_limit} = h_b))$ 
12: for all  $e = (p, r) \in E_{\text{dfs}}$  do
13:   if  $\rho_e = 1$  then
14:     if do_reduce is custom defined on  $r$  then
15:        $J_e \leftarrow \text{do\_reduce}(\text{prep\_for\_features}(D_r), k_r)$ 
16:     else
17:        $J_e \leftarrow \text{auto\_features}(r, k_r)$ 
18:     end if
19:   end if
20:    $D_p \leftarrow \text{LEFT\_JOIN}(D_p, J_e \text{ or } D_r)$ 
21:   if  $r$  is the label node or exposes a label field then
22:     if do_labels is custom defined on  $r$  then
23:        $L_r \leftarrow \text{do\_labels}(\text{prep\_for\_labels}(D_r))$ 
24:     else
25:        $L_r \leftarrow \text{default\_label}(\text{prep\_for\_labels}(D_r))$ 
26:     end if
27:      $D_p \leftarrow \text{LEFT\_JOIN}(D_p, L_r)$ 
28:   end if
29:    $D_p \leftarrow \text{do\_post\_join\_annotate}(D_p)$ 
30:    $D_p \leftarrow \text{do\_post\_join\_filters}(D_p)$ 
31:   if  $\alpha_e = 1$  then
32:      $D_p \leftarrow \text{do\_post\_join\_reduce}(D_p)$ 
33:   end if
34: end for
35: return hydrated parent-node relation
```

This execution pattern highlights three design choices central to GraphReduce. First, traversal is cardinality-aware and bottom-up, ensuring that lower-grain entities are reduced before joining into higher-grain parents. Second, reduction and label generation both support a hybrid custom-or-default execution path, preserving flexibility without breaking the graph-level contract. Third, point-in-time correctness is applied at feature-generation time and label-generation time through

`prep_for_features` and `prep_for_labels`, rather than being treated as a preprocessing detail at graph hydration time.

6.4 SQL automated feature generation: `sql_auto_features`

The SQL execution path uses a dynamic feature synthesis routine, `sql_auto_features`, to construct backend-specific aggregate expressions from a sampled view of the relation. The implementation proceeds in three stages: column-wise aggregation selection, optional time-series feature construction, and insertion of point-in-time filtering constraints [6].

At a high level, the routine first acquires a representative sample through `get_sample`, increasing sample size when sparsity is too high. It then infers candidate aggregation operators from physical or semantic type cues, a problem that relates to the broader literature on semantic data type detection such as Sherlock [11]. For a column c , let $\mathcal{A}(c)$ denote the set of aggregate operators selected for that column. The selection logic can be summarized as

$$\mathcal{A}(c) = \begin{cases} \{\text{count}\}, & \text{if } c \text{ is identifier-like,} \\ \Phi(\tau(c)), & \text{if a type map } \Phi \text{ is defined for type } \tau(c), \\ \{\text{sum}\}, & \text{if } c \text{ behaves like a binary categorical or boolean field,} \\ \emptyset, & \text{otherwise.} \end{cases} \quad (2)$$

The routine then emits SQL aggregate expressions of the form

$$f(c) \text{ AS alias}(c, f), \quad f \in \mathcal{A}(c),$$

subject to validity constraints on operator composition [6].

When the node is time-aware, `sql_auto_features` also generates temporal expressions. These include (i) a dialect-dependent measure of time since the last observed event, (ii) event counts within each configured historical window in `self.ts_periods`, and (iii) change ratios across adjacent windows, such as 30d/60d, to capture directional slope [6].

Let $T = \{\Delta_1, \dots, \Delta_m\}$ denote the configured historical windows. For a node grouped by reduce key k , the resulting time-series features can be described schematically as

$$\text{events}_\Delta(k) = \#\{r_i : r_i.k = k, t_{\text{cut}} - \Delta \leq t_i < t_{\text{cut}}\}, \quad (3)$$

$$\text{change}_{\Delta_j, \Delta_{j+1}}(k) = \frac{\text{events}_{\Delta_j}(k)}{\text{events}_{\Delta_{j+1}}(k)}, \quad (4)$$

$$\text{seconds_since_last}(k) = t_{\text{cut}} - \max_i \{t_i : r_i.k = k\}. \quad (5)$$

The final step is to incorporate point-in-time correctness directly into the generated SQL fragments. Specifically, the routine appends `WHERE` predicates driven by the graph-level `cut_date` and compute period so that all dynamic aggregates are computed only from temporally valid history. The SQL auto-feature path is therefore not a separate system layered on top of GraphReduce’s time semantics; it is one backend-specific realization of the same graph-level leakage constraints.

Algorithm 2 SQL automated feature generation with `sql_auto_features`

Require: Sample table S from node r , reduce key k , optional type-function map Φ

Require: Cut date t_{cut} , compute window W , historical periods $T = \{\Delta_1, \dots, \Delta_m\}$

Ensure: List of SQL aggregate/select operations

```
1: Build or enlarge sample  $S$  until sparsity is acceptable
2: Initialize empty list Ops
3: for all columns  $c$  in  $S$  do
4:   Determine physical or semantic type  $\tau(c)$ 
5:   Choose aggregate set  $\mathcal{A}(c)$  using heuristics or  $\Phi$ 
6:   for all  $f \in \mathcal{A}(c)$  do
7:     if composition is valid then
8:       Append SQL aggregate  $f(c)$  AS  $\text{alias}(c, f)$  to Ops
9:     end if
10:  end for
11: end for
12: if node is time-series aware then
13:   Append seconds-since-last-event expression
14:   for all  $\Delta \in T$  do
15:     Append count of events inside  $[t_{\text{cut}} - \Delta, t_{\text{cut}})$ 
16:   end for
17:   for all adjacent pairs  $(\Delta_j, \Delta_{j+1})$  do
18:     Append ratio feature  $\text{events}_{\Delta_j} / \text{events}_{\Delta_{j+1}}$ 
19:   end for
20: end if
21: Append point-in-time WHERE predicates implied by  $t_{\text{cut}}$  and  $W$ 
22: return Ops for compilation into the relation's SQL reduction query
```

Together, these details strengthen the central systems claim of GraphReduce: the SQL path is not merely a port of dataframe logic, but a rule-driven query-construction layer that maps the GraphReduce method contract into dialect-aware aggregate SQL while preserving lineage, temporal validity, and graph-structured execution.

7 Custom Nodes and Dynamic Nodes

GraphReduce supports both custom and dynamic execution modes. In the custom mode, developers subclass `GraphReduceNode` or a related backend-specific node and explicitly implement the required methods. This is appropriate when default aggregation primitives are insufficient or when business rules require entity-specific handling, such as removing duplicates, applying domain thresholds, or computing tailored statistical summaries.

In the automated mode, developers can instantiate `DynamicNode` objects and enable graph-level `auto_features`. In this configuration, GraphReduce automatically generates aggregate features on child tables, performs reductions, and joins the outputs back to the parent grain.

Custom and dynamic nodes are not separate systems. They are two implementation strategies under the same execution abstraction. This matters because it means automation does not require abandoning control, and customization does not require abandoning the graph-level programming

model.

Figure 3. Node prefixing and lineage preservation across joins

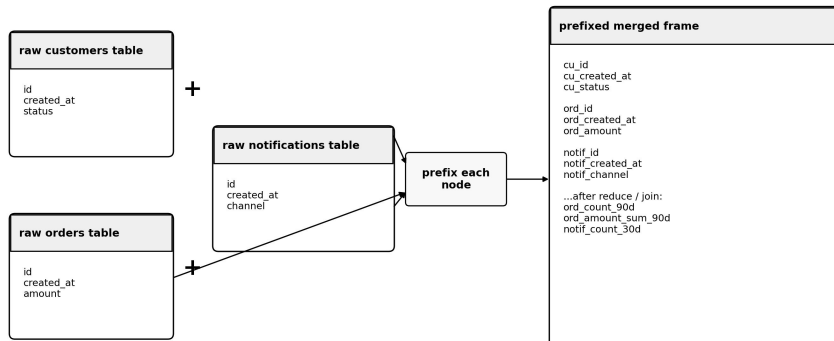


Figure 3: GraphReduce assigns each node a unique prefix so merged columns retain source-table lineage and avoid name collisions in the final parent-grain dataset. Prefixing makes downstream inspection easier by encoding provenance directly in feature names.

8 Graph Traversal, Joins, and Cardinality

Relational machine learning pipelines fail frequently at the boundary where lower-grain entities meet higher-grain outputs. Many join-heavy pipelines accidentally duplicate parent rows, distort counts, or conflate independent event histories. GraphReduce addresses this by making graph structure and reduction behavior explicit.

An edge in GraphReduce specifies the parent node, the relation node, the parent key, the relation key, and whether the relation must be reduced before joining. This encourages a cardinality-aware traversal in which lower-grain child tables are rolled up intentionally before being merged upward. In a customer-level pipeline, for example, order-products may first be reduced to orders, then orders reduced to customers, and only then joined into the final parent dataset.

9 Backend Interoperability

A major goal of GraphReduce is to separate logical feature construction from physical compute execution. This allows the same graph abstraction to run across multiple compute engines.

9.1 Dataframe backends

GraphReduce is designed to work with dataframe-oriented systems including pandas, Dask, Spark, and Daft [6]. Where APIs are sufficiently aligned, the system can swap the compute layer while preserving the graph definition and much of the feature logic.

9.2 SQL backends

GraphReduce also supports SQL execution through `SQLNode` and related backend-specific node types. Current documentation and examples include Databricks SQL, SQLite, Redshift, Athena, DuckDB, and Snowflake [6].

9.3 Dynamic query construction

SQL support is not just a thin wrapper around static query templates. GraphReduce provides abstractions for stackable SQL operations such as select expressions, where clauses, grouping logic, and aggregate functions. The system compiles these smaller operations into executable statements at runtime and pushes them down to the underlying engine.

Figure 4. Interoperability across dataframe and SQL compute layers

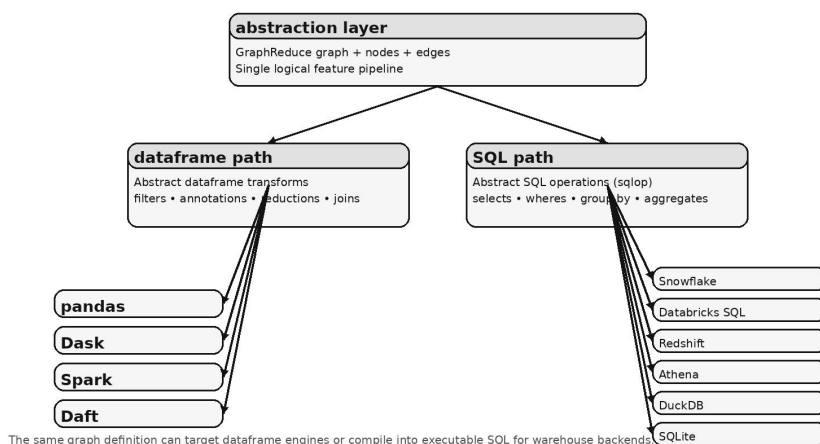


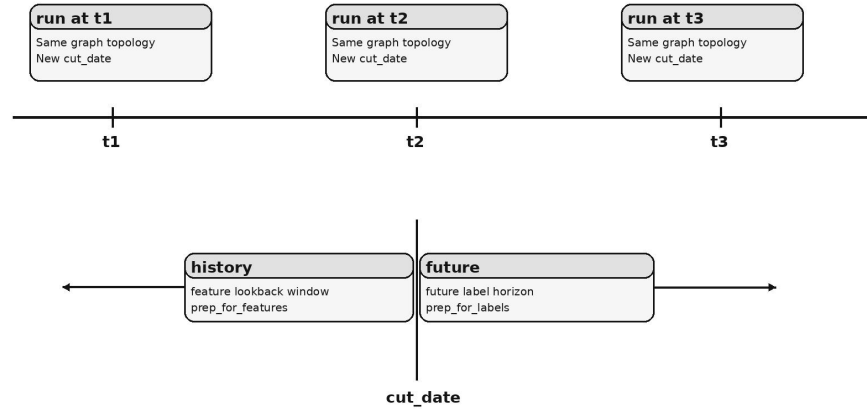
Figure 4: GraphReduce separates logical feature construction from physical execution. The same graph and node abstractions can compile into dataframe-native transformations or dynamically constructed SQL queries, allowing relational feature pipelines to remain portable across multiple compute backends.

10 Point-in-Time Correctness and Temporal Regeneration

Point-in-time correctness is a first-class property of GraphReduce. The system defines graph- and node-level timing controls including `cut_date`, `compute_period_val`, `compute_period_unit`, `label_period_val`, `label_period_unit`, and `date_key` on time-varying nodes.

This should be understood as both a correctness and a reproducibility property. Correctness follows because leakage-safe windows prevent future information from entering feature computation. Reproducibility follows because the same graph can be regenerated at arbitrary historical cut dates by changing top-level temporal parameters rather than rewriting the pipeline.

Figure 5. Point-in-time correctness and temporal regeneration



GraphReduce enforces a strict split between historical feature computation and future label generation. The same computational graph is reused across different cut dates by changing shared temporal parameters.

Figure 5: Point-in-time correctness is treated as a graph-level invariant. Features are generated only from records in the historical lookback window before the cut date, while labels are computed only in the forward horizon. The same computational graph can be regenerated at arbitrary historical windows by changing shared temporal parameters.

11 Post-Join Dependency Handling

Real feature pipelines often require logic that can only be executed after specific child datasets have been joined into a parent. GraphReduce supports this through method dependency control for post-join execution. For example, a parent node may need to compute a feature based on both its own timestamp and a child order timestamp, or to filter rows based on the joint presence of multiple child summaries. By allowing post-join methods to declare dependencies on specific nodes, GraphReduce makes this multi-table sequencing explicit instead of burying it in ad hoc procedural code.

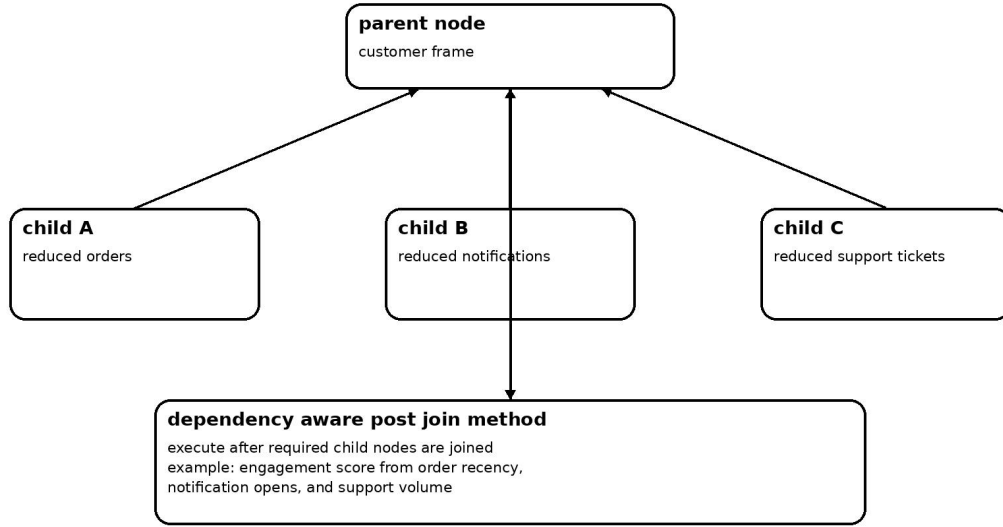


Figure 6: GraphReduce supports dependency-aware post-join execution for transformations that require merged context from specific child nodes. By declaring method requirements, a node can defer post-join annotations or filters until the relevant child data has been joined.

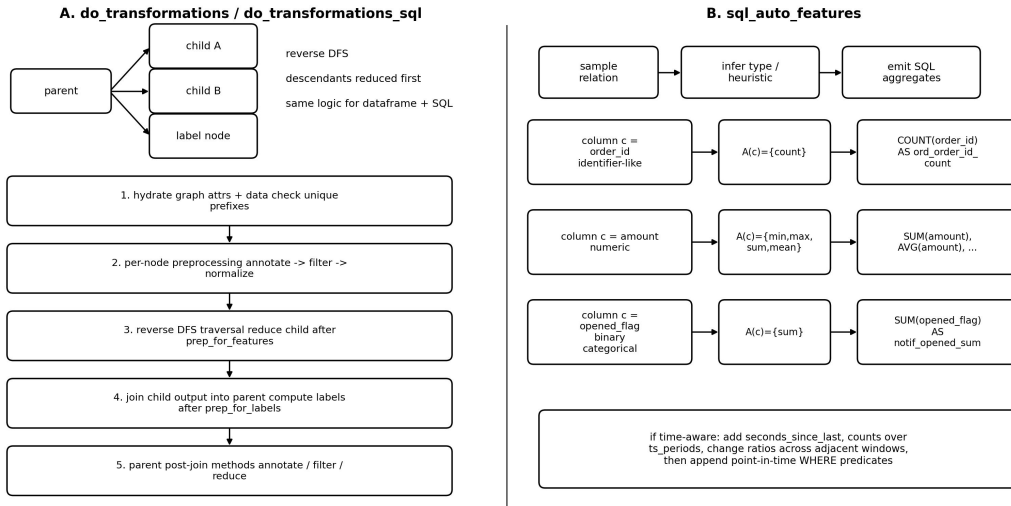


Figure 7: Concrete execution of GraphReduce transformations and SQL auto-feature generation. Left: graph execution is bottom-up and cardinality-aware. Right: SQL auto-features convert typed columns and time windows into dialect-aware aggregate expressions while preserving prefix lineage and `cut_date` filtering.

12 Empirical Evaluation on RelBench

We include a compact empirical section to situate GraphReduce as a practical relational feature-generation layer on standardized RelBench tasks. RelBench is a benchmark for predictive modeling over relational databases, and it introduces both benchmark datasets and a family of relational deep learning baselines [8]. The results in Tables 1 and 2 are reproduced from the official GraphReduce documentation and should be interpreted as the performance of GraphReduce-generated feature frames paired with a downstream CatBoost model [6, 10]. This framing is important: GraphReduce is evaluated here as the relational feature-generation layer rather than as a predictive model in isolation.

12.1 Published Performance Summary

In Tables 1 and 2, *RDL* denotes the relational deep learning baselines reported in the RelBench benchmark paper, where heterogeneous graph neural models are paired with deep tabular encoders over relational databases [8]. *KumoRFM-1* and *KumoRFM-2* denote the two relational foundation model generations reported in the recent KumoRFM-2 paper [9]. Unless otherwise stated, the KumoRFM-2 paper reports predictions from the base model through in context learning without additional task specific training or fine tuning. The *Data Scientist* column corresponds to the manual feature engineering baseline originating from RelBench [8]. The GraphReduce plus CatBoost column is populated from the current GraphReduce RelBench run suite used for this paper.

Across the 21 task summary reproduced here, GraphReduce results are available for 19 tasks. On those 19 comparable tasks, GraphReduce plus CatBoost is the strongest overall method on 4 tasks: rel-avito user-visits, rel-avito user-clicks, rel-avito ad-ctr, and rel-stack post-votes. That is not an overwhelming win count. However, the broader result is still provocative. Relative to the relational foundation model baselines reported in KumoRFM-2, GraphReduce plus CatBoost outperforms KumoRFM-1 on 12 of 19 comparable tasks and KumoRFM-2 on 8 of 19 comparable tasks. This comparison is notable because GraphReduce plus CatBoost is not a large black box architecture and is not tuned through the kind of task specific customizations, filters, and exclusions that are commonly required in real production data lakes. Instead, it remains a fully customizable relational signal compression system that can sit alongside existing MLOps infrastructure, execute inside familiar data systems, and feed arbitrary downstream learners such as CatBoost, PySpark ML, or warehouse native ML tools. The result is therefore less about dominating the benchmark outright and more about showing that an explicit, production ready, and interpretable systems approach can remain competitive with state of the art relational deep learning and relational foundation model baselines while preserving flexibility that matters in real enterprise workflows.

Table 1: RelBench entity classification summary combining baselines reported in KumoRFM-2 with GraphReduce plus CatBoost results from the current GraphReduce run suite. Higher is better for ROC-AUC. Missing values indicate that the corresponding baseline or GraphReduce result was not reported.

Dataset	Task	LightGBM	Data Scientist	RDL	LLM	KumoRFM-1	KumoRFM-2	GraphReduce + CatBoost
rel-amazon	user-churn	52.22	67.60	70.42	62.55	67.29	69.27	69.83
rel-amazon	item-churn	62.54	81.80	82.81	73.41	79.93	82.17	82.49
rel-avito	user-visits	53.05	–	66.20	53.36	64.85	69.41	82.99
rel-avito	user-clicks	53.60	–	65.90	54.07	64.11	67.42	78.95
rel-event	user-repeat	53.05	–	76.89	53.36	76.08	81.66	64.96
rel-event	user-ignore	79.93	–	81.62	68.65	89.20	90.83	60.47
rel-fl	driver-dnf	68.86	69.80	72.62	80.03	82.41	84.59	71.51
rel-fl	driver-top3	73.93	82.40	75.54	87.11	91.07	92.18	82.46
rel-hm	user-churn	55.21	69.00	69.88	63.81	67.71	69.27	69.52
rel-stack	user-engagement	63.39	90.30	90.59	81.23	87.09	89.40	89.12
rel-stack	user-badge	63.43	86.20	88.86	79.99	80.00	87.15	84.42
rel-trial	study-outcome	70.09	72.00	68.60	59.17	70.79	72.03	61.57

Table 2: RelBench entity regression summary combining baselines reported in KumoRFM-2 with GraphReduce plus CatBoost results from the current GraphReduce run suite. Lower is better for MAE. Missing values indicate that the corresponding GraphReduce result was not reported.

Dataset	Task	LightGBM	Data Scientist	RDL	KumoRFM-1	KumoRFM-2	GraphReduce + CatBoost
rel-amazon	user-ltv	16.783	13.928	14.313	16.161	13.921	14.02
rel-amazon	item-ltv	60.569	41.122	50.053	55.254	46.992	42.71
rel-avito	ad-ctr	0.041	–	0.041	0.035	0.034	0.031
rel-event	user-attendance	0.264	–	0.258	0.264	0.235	0.293
rel-fl	driver-position	4.170	3.963	4.022	2.747	2.854	3.59
rel-hm	item-sales	0.076	0.036	0.056	0.040	0.034	0.036
rel-stack	post-votes	0.068	0.065	0.065	0.065	0.064	0.063
rel-trial	study-adverse	44.011	40.581	44.473	58.231	43.293	–
rel-trial	site-success	0.425	0.407	0.400	0.417	0.433	–

12.2 CatBoost Configuration and Evaluation Protocol

The example task code in the GraphReduce examples directory uses a consistent CatBoost setup across task families [6, 10]. Classification tasks use `CatBoostClassifier` with depth 8, learning rate 0.05, random seed 42, and AUC-based evaluation. Regression tasks use `CatBoostRegressor` with comparable depth and learning-rate settings, with task-specific loss and metric choices. In the examples, GraphReduce constructs the flattened parent-grain frame, after which CatBoost is trained as a downstream tabular learner on the generated features. This separation matches the intended role of GraphReduce in the empirical story of the paper: the system is evaluated as the relational feature-generation layer that supplies a conventional downstream model.

13 Relationship to Prior Work

13.1 Automated feature engineering systems

GraphReduce is influenced by prior systems for automatic feature generation over relational data, especially Deep Feature Synthesis and One Button Machine [2, 3]. These systems helped establish that relational structure can be exploited to generate useful aggregate features automatically.

GraphReduce shares the goal of reducing manual feature engineering effort, but differs in several ways. First, it places stronger emphasis on interoperability across both dataframe and SQL execution substrates. Second, it treats custom node-level logic as a first-class design goal rather than an escape hatch. Third, it foregrounds point-in-time correctness and graph regeneration across arbitrary windows. Fourth, it frames execution around a stable node-method contract and deterministic graph traversal rather than only a library of compositional primitives.

13.2 Featuretools and practical automated synthesis

Featuretools helped popularize automated feature engineering in practice and made relational feature synthesis more accessible to practitioners [4]. GraphReduce is philosophically aligned with that movement, but is aimed more directly at explicit graph execution, backend portability, and production-oriented pipeline design where customization and leakage-safe reruns are often mandatory.

13.3 Warehouse-native transformation and feature stacks

A modern buyer is not comparing GraphReduce only to research systems or older automated feature engineering libraries. In practice, they are often comparing it to warehouse-native transformation and feature workflows built with dbt, Snowflake, and Databricks [12, 13, 14]. These systems are operationally important and increasingly support feature engineering directly where the data lives. However, their primary abstractions are different. dbt emphasizes modular SQL transformation and documentation rather than a graph-native execution model for relational feature generation [12]. Snowflake and Databricks expose feature-store and feature-engineering capabilities inside the warehouse or lakehouse, but they do not by themselves provide the same portable graph abstraction with interchangeable custom and dynamic node behavior, explicit parent-child reduction semantics, and graph-level point-in-time regeneration that GraphReduce emphasizes [13, 14]. GraphReduce is therefore best understood as complementary to warehouse-native stacks: it can target those environments while contributing a more explicit execution model for relational signal compression.

13.4 Graph-based relational learning approaches

Graph-based feature synthesis and graph neural approaches operate on a related problem space but differ substantially in modeling assumptions and execution style [5]. Many graph-learning systems emphasize representation learning directly over relational structure, whereas GraphReduce is designed to produce explicit parent-grain training frames through interpretable joins, rollups, and time-windowed aggregations. The system is therefore best viewed as infrastructure for relational data preparation rather than an end-to-end graph representation learner.

13.5 Relational prediction with foundation models

A recent and especially relevant paper is RDBLearn: Simple In-Context Prediction Over Relational Databases [7]. RDBLearn starts from the observation that many real-world prediction tasks live in relational databases rather than single flat tables. It proposes a simple recipe: automatically featurize each target row using relational aggregations over linked records, materialize the resulting augmented table, and run an off-the-shelf tabular foundation model on that flattened representation.

This line of work is highly complementary to GraphReduce rather than contradictory to it. It strengthens the argument that relational tasks still benefit from explicit relational aggregation and flattening before downstream prediction. GraphReduce is differentiated by focusing on the execution abstraction that produces those leakage-safe parent-grain frames across multiple dataframe and SQL backends, with explicit graph traversal, developer-defined node logic, dynamic automated operations, and point-in-time regeneration as first-class concerns.

13.6 MapReduce as conceptual inspiration

MapReduce is relevant here as a conceptual ancestor in one narrow sense: it demonstrated the value of a compact programming model that hides execution complexity behind a small set of abstractions [1]. GraphReduce adopts that spirit for a different problem. It offers a simplified abstraction for graph-shaped relational feature execution while leaving the underlying compute engine free to vary. The relationship is one of inspiration, not functional equivalence.

14 Contributions

This paper makes the following contributions. First, it frames multi-table feature generation as relational signal compression, a systems problem in which predictive signal distributed across many related tables is compressed into a leakage-safe parent-grain representation. Second, it presents a graph-based execution abstraction in which nodes, edges, and deterministic traversal define a reusable computational plan for relational feature generation. Third, it describes a hybrid implementation model that supports both developer-authored node logic and dynamic automated feature generation under the same abstract method contract. Fourth, it shows how GraphReduce separates logical feature construction from physical execution across dataframe and SQL backends. Fifth, it elevates point-in-time correctness into a graph-level property, enabling reproducible regeneration of the same computational graph across arbitrary historical windows.

15 Limitations and Future Work

GraphReduce is not a universal solution to all relational learning problems. Its abstraction is strongest when the end goal is an explicit parent-grain frame for supervised learning, scoring, or analytics. It is less naturally suited to problems that benefit from fully end-to-end representation learning over graphs without intermediate feature materialization.

In addition, backend portability does not imply that all compute engines expose identical performance characteristics or implementation complexity. Some backends may require additional dialect-specific support or optimization. Similarly, automated feature generation provides useful baselines but does not eliminate the need for custom domain logic in many production systems.

Promising future directions include richer semantic typing for more expressive dynamic operations, stronger online or incremental execution patterns, deeper integration with metadata inference systems, improved cost-based planning across backends, and broader benchmark-based evaluation on large relational datasets.

16 Conclusion

The central challenge of practical machine learning over relational data is still structural. Enterprises do not store predictive information in one clean matrix; they store it across many entities, keys, and event histories. Models, by contrast, still expect a compact training frame at a single grain. GraphReduce addresses this mismatch by treating relational feature generation as an executable graph problem.

The key idea is not merely automation. It is the combination of a stable operation contract, cardinality-aware traversal, support for both handwritten and dynamic logic, portability across dataframe and SQL systems, and point-in-time correctness as a first-class invariant. Together these properties make GraphReduce more than a feature library. They make it an abstraction layer for relational signal compression.

As tabular models and relational AI systems continue to improve, the upstream need for interpretable, reproducible, leakage-safe relational data preparation is unlikely to disappear. If anything, stronger downstream models increase the value of a principled upstream system that can repeatedly generate trustworthy inputs. GraphReduce is intended as such a system.

Acknowledgments

The author thanks the anonymous reviewers for their thoughtful comments and suggestions, which helped improve the clarity and presentation of this paper.

References

- [1] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.
- [2] J. M. Kanter and K. Veeramachaneni. Deep Feature Synthesis: Towards Automating Data Science Endeavors. In *2015 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2015.
- [3] H. T. Lam, J.-M. Thiebaut, M. Sinn, B. Chen, T. Mai, and O. Alkan. One Button Machine for Automating Feature Engineering in Relational Databases. *arXiv preprint arXiv:1706.00327*, 2017.
- [4] Featuretools Development Team. Featuretools documentation and project materials. Available at the Featuretools project website, accessed 2026.
- [5] H. Zhang, Q. Gan, D. Wipf, and W. Zhang. GFS: Graph-Based Feature Synthesis for Prediction over Relational Databases. 2023.
- [6] W. Madrigal. GraphReduce documentation, repository, examples, implementation materials, and published RelBench results. Official GraphReduce documentation site and GitHub repository, accessed April 1, 2026.
- [7] Y. Zhang, L. Xu, Q. Gan, D. Wipf, and M. Wang. RDBLearn: Simple In-Context Prediction Over Relational Databases. *arXiv:2602.18495*, 2026.

- [8] J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec. RelBench: A Benchmark for Deep Learning on Relational Databases. *arXiv:2407.20060*, 2024.
- [9] V. Hudovernik, F. López, V. Kocijan, A. Nitta, J. E. Lenssen, J. Leskovec, and M. Fey. KumoRFM-2: Scaling Foundation Models for Relational Learning. Kumo AI Research Paper, 2026.
- [10] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.
- [11] M. Hulsebos, K. Hu, M. Bakker, E. Zraggen, A. Satyanarayan, T. Kraska, C. Demiralp, and C. Hidalgo. Sherlock: A Deep Learning Approach to Semantic Data Type Detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1500–1508, 2019.
- [12] dbt Labs. What is dbt? and dbt documentation. dbt Developer Hub, accessed 2026.
- [13] Snowflake. Snowflake Feature Store overview and Snowflake ML overview. Official Snowflake documentation, accessed 2026.
- [14] Databricks. Databricks Feature Store and Feature Engineering in Unity Catalog. Official Databricks documentation, accessed 2026.
- [15] A. Hagberg, P. Swart, and D. Schult. Exploring Network Structure, Dynamics, and Function Using NetworkX. Los Alamos National Laboratory, 2008.